Федеральное государственное бюджетное образовательное учреждение высшего образования
«Херсонский государственный педагогический университет»

Институт информационных технологий

Кафедра программной инженерии и информационных технологий

ПРАКТИЧЕСКАЯ РАБОТА
по дисциплине:
«Системы обработки больших данных»

на тему:
«Среднее и повторная обработка
Частичная агрегация данных»

Выполнил:
Студент гр.: 12-25РПм
Трифонов Д.С.

Проверил
Преподаватель:
Гуцол Д.В.

Оценка:_____
Дата: 13.10.2026

Херсон, 2026

ПРАКТИЧЕСКАЯ РАБОТА

Цель работы

Выполнить два практикума локальной последовательной модели на Python: посчитать частичные количества и суммы, свести их по регионам и сверить каждый регион с прямым расчётом; затем отличить невзвешенное среднее от верного, обнаружить повтор результата задачи и получить прежний итог после контроля повторов.

Среда: Windows, Python 3.14, дополнительные пакеты не использовались. Hadoop не устанавливался, ускорение кластера не измерялось. Команды запускались из папки проекта резервным способом, указанным в практикуме: py -3 lab.py <шаг>. Исходный CSV и контрольные описания не изменялись.

1. Частичная агрегация данных

Шаг A0. Подготовка среды

В папке проекта рядом находятся lab.py, REPORT.txt, data/clean_events.csv и 00_CHECK_ENV.cmd. Проверка среды сообщила, что интерпретатор доступен и в источнике 576 строк.



Рисунок 1 — Проверка среды первой пары

Шаг A1. Разделение на три части

Команда partition разложила исходные события по трём разделам без фильтра и без агрегации. Контрольная строка: SOURCE_ROWS=576; part_0: rows=206; part_1: rows=178; part_2: rows=192.

Ручная проверка сохранения строк: 206 + 178 = 384; 384 + 192 = 576. Потерь нет.

Количество покупок на этом шаге ещё не получено. Раздел хранит все события: успешные покупки, просмотры, клики и покупки со статусами failed и cancelled. Числа 206, 178 и 192 — это размеры частей, а не число логических покупок. Фильтр применяется только на следующем шаге.

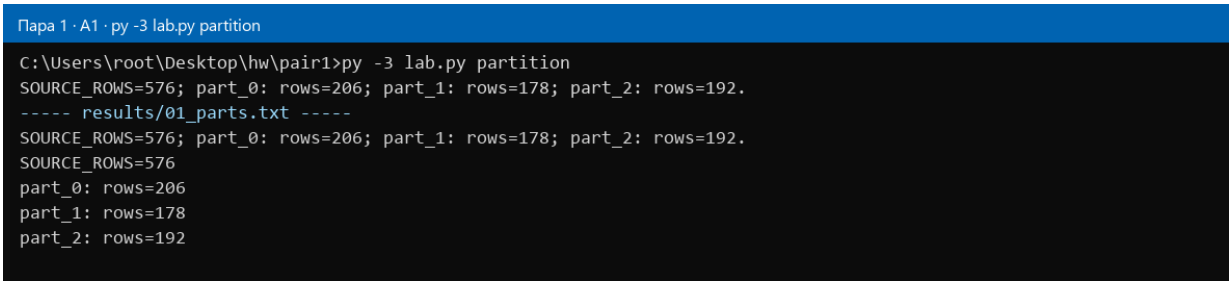


Рисунок 2 — Создание разделов и файл results/01_parts.txt

Шаг A2. Частичные агрегаты

Команда local читает каждый раздел отдельно и записывает partials/part_0.json, part_1.json и part_2.json. Условие отбора, найденное в функции summarize: строка учитывается, только если тип события равен purchase и статус равен success. Остальные строки в count и sum_cents не входят.

Раздел	Успешные покупки	Сумма, руб.
part_0	38	99596.53
part_1	27	79029.15
part_2	25	66382.45

Назначение полей частичного JSON:

task_id — идентификатор логической задачи, а не имя файла. У part_0 это значение part_0.

groups — список итогов по регионам внутри одного раздела.

count — число успешных покупок региона в этом разделе.

sum_cents — сумма тех же покупок в копейках. Деление на 100 переводит её в рубли.

dataset_sha256 — отпечаток исходного CSV, чтобы частичный результат нельзя было молча отнести к другому набору.

sha — отпечаток списка groups. Он отличает идентичный повтор от конфликта содержимого.

Пара 1 · A2 · py -3 lab.py local

```
C:\Users\root\Desktop\hw\pair1>py -3 lab.py local
part_0: 38 покупок, 99596.53 руб.; part_1: 27, 79029.15; part_2: 25, 66382.45.
----- results/02_local.txt -----
part_0: 38 покупок, 99596.53 руб.; part_1: 27, 79029.15; part_2: 25, 66382.45.

part_0: purchases=38; amount=99596.53
part_1: purchases=27; amount=79029.15
part_2: purchases=25; amount=66382.45
----- partials/part_0.json -----
{
  "task_id": "part_0",
  "dataset_sha256": "c37ac7992441c34b51866648cacd55172327cfe934fa4434a0ade2e5e53f0784",
  "sha": "81738dda2a69041dbcd1958f8ce1d5f4cb9560b29d86ec92d3e0698030cc132a",
  "groups": [
    {
      "region": "Запорожская область",
      "count": 7,
      "sum_cents": 1850000
    },
    {
      "region": "Николаевская область",
      "count": 8,
      "sum_cents": 2100000
    },
    {
      "region": "Одесская область",
      "count": 7,
      "sum_cents": 1900000
    },
    {
      "region": "Республика Крым",
      "count": 6,
      "sum_cents": 1609653
    },
    {
      "region": "Херсонская область",
      "count": 10,
      "sum_cents": 2500000
    }
  ]
}
```

Рисунок 3 — Частичные агрегаты и поля part_0.json

Шаг A3. Объединение по регионам

Команда reduce складывает count и sum_cents трёх частичных результатов по ключу группировки — названию региона. Получено пять регионов. Общий итог: purchases=90; amount=245008.13. Машинный результат записан в results/aggregate.json.

Регион	Покупки	Сумма, руб.
Херсонская область	21	57000.00
Николаевская область	18	49000.00
Запорожская область	18	49500.00
Одесская область	17	46411.60
Республика Крым	16	43096.53

Итого	90	245008.13

Ручной расчёт Херсонской области по трём файлам partials/part_N.json.

В part_0: count = 10, sum_cents = 2500000. В part_1: count = 6, sum_cents = 1800000. В part_2: count = 5, sum_cents = 1400000. Область есть во всех трёх частичных результатах, нулевых вкладов нет.

Сложение количеств: 10 + 6 + 5 = 21.

Сложение копеек целыми числами: 2500000 + 1800000 + 1400000 = 5700000.

Перевод в рубли: 5700000 / 100 = 57000.00. Строка общего результата совпадает с этим расчётом.

Сложение трёх разделов для всего набора:

Количество: 38 + 27 + 25 = 90.

Сумма: 99596.53 + 79029.15 = 178625.68; 178625.68 + 66382.45 = 245008.13 руб.

Ключ группировки — регион. Сводить можно именно количество и сумму в копейках: эти операции ассоциативны и коммутативны, порядок разделов на итог не влияет. Среднее на частичных результатах так сводить нельзя, это показывает второй практикум.

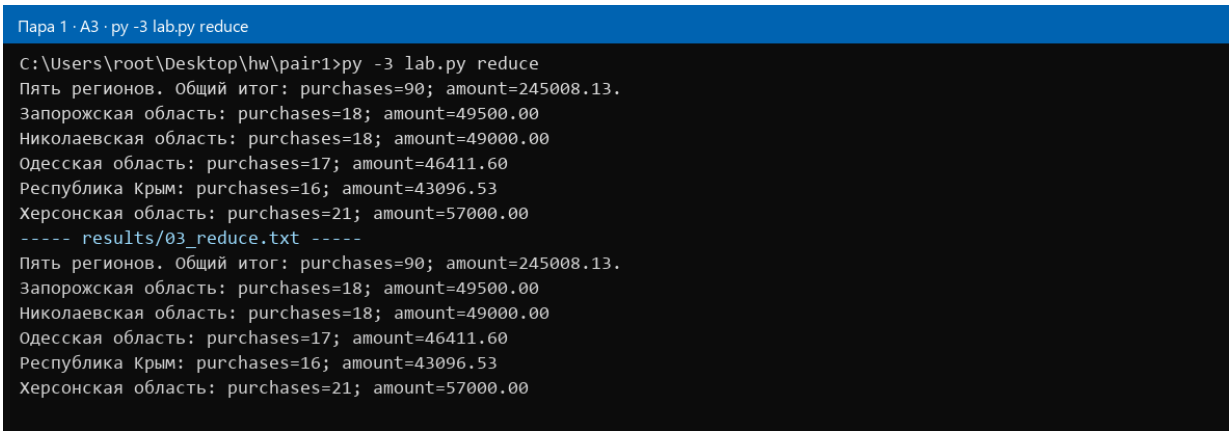


Рисунок 4 — Сведение пяти регионов

Шаг A4. Сверка с исходными данными

Команда verify заново отбирает успешные покупки прямо из clean_events.csv и сравнивает с aggregate.json количество и точную сумму каждого региона. Результат: PASS. Logical purchases=90; amount=245008.13.

Проверка всех регионов сильнее проверки одной общей суммы. Если один регион завышен, а другой занижен на ту же величину, общая сумма останется прежней, но ошибка распределения сохранится. Порегионное сравнение такое погашение обнаруживает.

Условие отбора словами: в функцию summarize попадает событие, которое одновременно является покупкой (purchase) и имеет успешный статус (success). Любой другой тип или статус отбрасывается до суммирования.

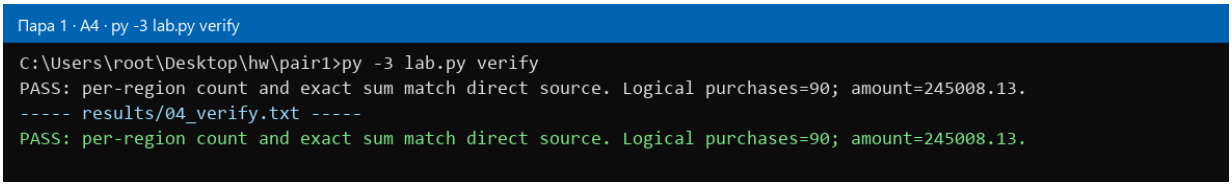


Рисунок 5 — Прямая сверка по регионам

Шаг A5. Отчёт и архив первой пары

REPORT.txt заполнен расчётами и пояснениями в кодировке UTF-8. Упаковка завершилась строкой READY и создала submission/BigData4_pair1_result.zip. В архив вошли отчёт, исходный CSV, контрольное описание, разделы, частичные JSON и папка results. Весь стартовый каталог повторно не упаковывался.



Рисунок 6 — Архив результата первой пары

2. Среднее и повторная обработка

Вторая пара начинает с уже подготовленных parts и partials. Результаты первой пары не переносились: в проекте свои разделы, свои частичные JSON и собственный архив.

Шаг Б0. Проверка среды

Повторная проверка показала тот же источник: Python OK; source rows=576.



Рисунок 7 — Проверка среды второй пары

Шаг Б1. Два средних

Команда averages выводит WRONG_UNWEIGHTED_MEAN=2734.42 и CORRECT_WEIGHTED_MEAN=2722.31. Верная формула: 245008.13 / 90 = 2722.31.

Локальные количества, суммы и средние, посчитанные из неокруглённых сумм:

Раздел	Покупки	Сумма, руб.	Среднее, руб.
part_0	38	99596.53	99596.53 / 38 = 2620.9613…
part_1	27	79029.15	79029.15 / 27 = 2927.0055…
part_2	25	66382.45	66382.45 / 25 = 2655.2980…

Невзвешенное среднее получается усреднением трёх локальных средних, как будто разделы равноправны: (2620.9613157895 + 2927.0055555556 + 2655.298) / 3 = 2734.4216…, после округления до двух знаков 2734.42. Программа считает эту величину по неокруглённым средним. Восстанавливать общую сумму из уже округлённых средних нельзя.

Верное среднее взвешивает раздел числом покупок: (99596.53 + 79029.15 + 66382.45) / (38 + 27 + 25) = 245008.13 / 90 = 2722.3125… = 2722.31.

Числа 206, 178 и 192 весами служить не могут. Это количества всех строк раздела, включая события, которые фильтр purchase + success отбрасывает. Вес покупки — единица на каждую успешную покупку, поэтому вес раздела равен 38, 27 или 25. Подстановка размеров разделов отвечала бы на другой вопрос: она смешивала бы покупки с просмотрами и отказами.

Ручной учебный пример из задания.

Дано: одна покупка на 100 руб. и три покупки на общую сумму 900 руб.

Среднее группы из трёх покупок: 900 / 3 = 300 руб.

Невзвешенное среднее двух групп: (100 + 300) / 2 = 400 / 2 = 200 руб. Группа из одной покупки получила такой же вес, как группа из трёх.

Верный итог считается по исходным суммам и количествам: (100 + 900) / (1 + 3) = 1000 / 4 = 250 руб.

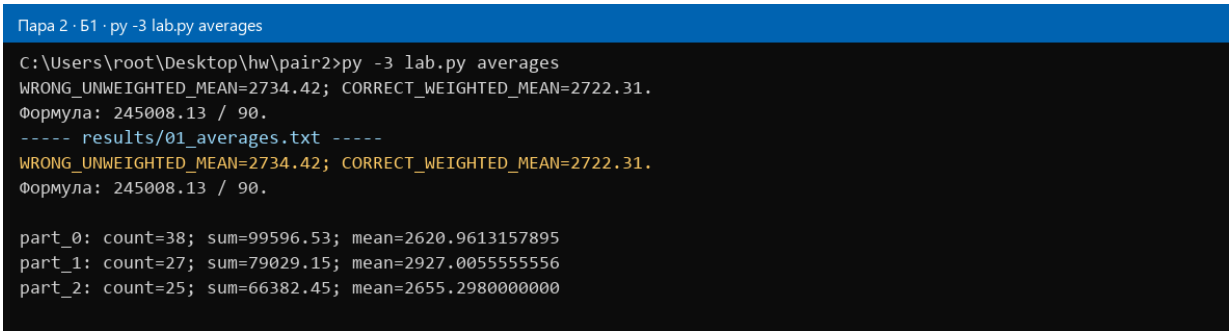


Рисунок 8 — Невзвешенное и взвешенное средние

Шаг Б2. Повтор результата

Команда repeat создала partials/part_0_retry.json. Контрольная строка: FILES=4; UNIQUE_TASKS=3. NAIVE_COUNT=128; NAIVE_AMOUNT=344604.66.

part_0.json и part_0_retry.json открыты как текст. У них одинаковый task_id (part_0), одинаковый dataset_sha256, одинаковый sha и одинаковый список groups. Это не новая порция покупок, а вторая копия уже посчитанного результата.

Наивный подсчёт без проверки повтора:

Количество: 90 + 38 = 128.

Сумма: 245008.13 + 99596.53 = 344604.66 руб.

Лишние 38 покупок и 99596.53 руб. — это повторный вклад part_0. Исходный CSV не изменялся: задвоение появилось только потому, что сведение прочитало четыре файла и не заметило, что логических задач по-прежнему три.

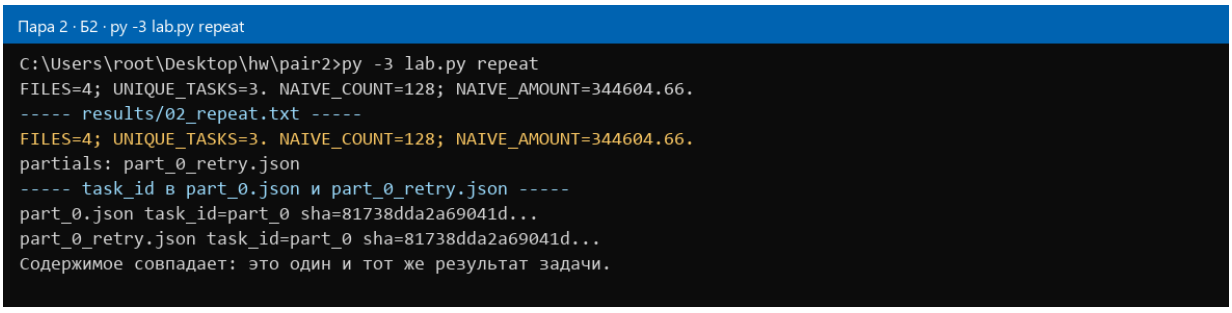


Рисунок 9 — Повтор part_0 и наивный итог

Шаг Б3. Ожидаемая остановка

Строгое объединение остановилось с сообщением:

EXPECTED STOP: Duplicate task_id=part_0. Strict reducer refused double counting.

Это полезный отказ. Повтор логической задачи опаснее обычной ошибки округления: он незаметно увеличивает и количество, и сумму. Строгий reducer не выбирает «какую копию оставить», а прекращает сведение, чтобы двойной учёт нельзя было принять за успех. Команда guard перехватывает именно этот ожидаемый отказ и сама завершается успешно. Следующий шаг — safe, программу переустанавливать не нужно.

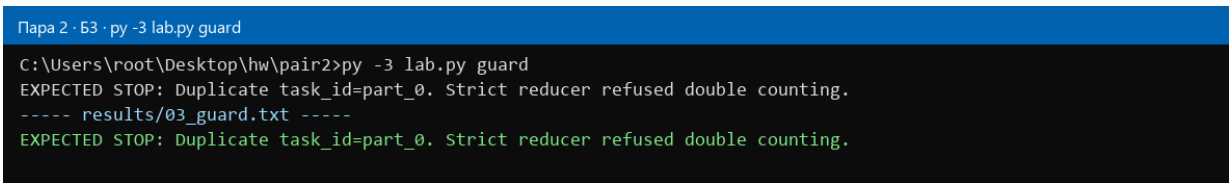


Рисунок 10 — Ожидаемая остановка строгого объединения

Шаг Б4. Идентичный повтор учитывается один раз

Команда safe вывела IGNORED_IDENTICAL_REPEATS=1; PURCHASES=90; AMOUNT=245008.13; MEAN=2722.31 и строку PASS: every region matches the direct reference. Повторный запуск 04_SAFE оставил тот же итог: исключение идентичной копии идемпотентно.

Сравнение с наивным итогом шага Б2: 128 покупок и 344604.66 руб. против восстановленных 90 покупок и 245008.13 руб. Разница равна ровно одному лишнему вкладу part_0.

Три разных ситуации:

Идентичный повтор задачи — тот же task_id и тот же sha содержимого. Это одна и та же работа, доставленная дважды. Её учитывают один раз.

Конфликт результатов — тот же task_id, но другой sha или другой отпечаток набора. Какую версию выбрать, неизвестно, поэтому такой файл нельзя отбросить как повтор. Сведение должно остановиться.

Две настоящие покупки с одинаковой ценой — разные строки исходных данных. У них разные события, они обе проходят фильтр purchase + success и обе входят в count. Совпадение цены не делает одну из них повтором задачи.

Одного имени файла для распознавания повтора мало. part_0_retry.json называется иначе, чем part_0.json, но несёт тот же task_id. В load_partials идентичность проверяется по task_id, dataset_sha256 и sha: имя лишь указывает, откуда прочитан документ. Без task_id повтор под новым именем был бы посчитан второй раз, а без sha нельзя отличить точную копию от испорченного результата с тем же идентификатором.

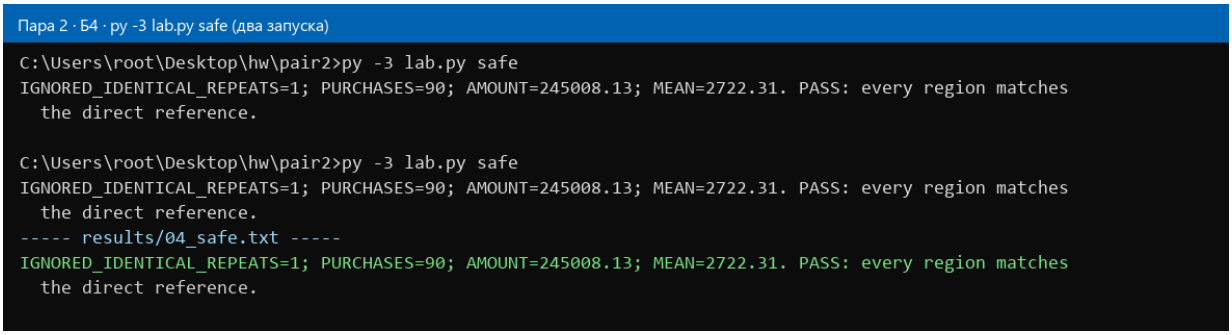


Рисунок 11 — Безопасное сведение и повторный запуск

Шаг Б5. Архив второй пары

После заполнения REPORT.txt упаковка выдала READY и создала submission/BigData4_pair2_result.zip. В архиве отчёт, исходный CSV, контрольное описание, частичные JSON, включая идентичный повтор, и results.

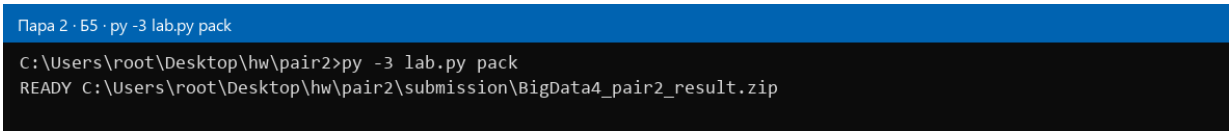


Рисунок 12 — Архив результата второй пары

Выводы

1. 576 событий разделены на части 206, 178 и 192 без потери строк. Число покупок на этапе разделения не возникает: фильтр ещё не применён.

2. После отбора успешных покупок получены частичные итоги 38 / 99596.53, 27 / 79029.15 и 25 / 66382.45. Сведение по региону даёт 90 покупок и 245008.13 руб. Херсонская область сверена вручную: 21 покупка и 57000.00 руб.

3. Порегионная сверка с прямым проходом по CSV подтвердила и количества, и точные суммы. Общей суммы для такой проверки недостаточно.

4. Среднее локальных средних равно 2734.42 и не совпадает с верным 245008.13 / 90 = 2722.31. Вес раздела — число покупок, а не число всех строк.

5. Повтор файла с тем же task_id наивно раздувает итог до 128 покупок и 344604.66 руб. Строгое объединение обязано остановиться. Идентичную копию можно учесть один раз; конфликт содержимого и две реальные покупки одной цены — это другие случаи.

Граница модели: вычисления локальные и последовательные. Они показывают, какие величины можно сводить по частям (count и sum_cents) и какие нельзя (среднее), но не измеряют отказоустойчивость и ускорение кластера Hadoop.

Затруднений при выполнении этапов не возникло. Все контрольные строки совпали с указанными в практикумах.